

Aula 12: Recursividade

Luís Felipe

UFF

05 de Novembro de 2025

Começando do começo...

Em computação existe uma técnica muito utilizada para construirmos algoritmos.

Esta técnica é denominada **divisão e conquista** (**divide and conquer**) e consiste em: determinar a solução de problema buscando sua solução através da solução de problemas menores.

Curiosidade: Termo clássico em estratégias de guerras. Gere uma divisão de seus inimigos para aí sim, conquistá-los. Estratégia utilizado por Júlio César e termo conhecido desde a Antiguidade.

Em computação: Às vezes conseguimos soluções mais eficientes quando pensamos dessa forma.

Exemplo: Problema de busca numa lista ordenada:

Entrada: 2 5 8 10 15 20 25

- **Saída:** Posição do 16.

Possível algoritmo: Percorra do início até o fim da lista, indo para a posição seguinte se elemento for menor ao buscado.

Resposta: 6 passos para concluir que 16 não se encontra.

Algoritmo de Busca Binária: Veja quem está no meio da lista. Se o elemento que buscamos for menor, então descarte a segunda metade e faça a mesma análise para a primeira metade. Analogamente, se elemento que buscamos for maior que o meio, descarte a primeira metade e faça a mesma análise para a segunda metade.

Resposta: 3 passos para concluir que 16 não se encontra.

Exemplo: Problema de ordenação de uma lista:

- **Entrada:** 25 5 10 2 20 8 15

- **Saída:** transformação em 2 5 8 10 15 20 25

Algoritmo MergeSort: divida ao meio; ordene cada metade (**recursivamente**); compõe soluções das metades. Com essa técnica conseguimos melhorar a complexidade de ordenação de $O(n^2)$ para $O(n \log n)$ (veremos em mais detalhe na próxima aula).

Definição

- Para uma sequência de números $(a_0, a_1, \dots, a_n, \dots)$, uma equação que relaciona o termo geral a_n a um ou mais de seus predecessores na sequência, para qualquer n , é dita uma **relação de recorrência** ou **equação de diferenças**.
- O fato de a_n ser escrito em função de predecessores faz com que o conhecimento de um ou mais termos iniciais da sequência seja necessário para seu cálculo. Tais elementos são denominados **condições iniciais** ou **condições de fronteira**.

Recursividade

- Uma função é chamada **recursiva** quando possui no seu corpo uma chamada a ela própria.
- Um exemplo clássico é o cálculo da função fatorial de um número natural.

$$n! = \begin{cases} 1, & \text{se } n = 0 \quad (\text{caso base}) \\ n \cdot (n - 1)!, & \text{se } n > 0 \quad (\text{expressão de recorrência}) \end{cases}$$

Exemplo: $3! = 3 \times 2! = 3 \times 2 \times 1! = 3 \times 2 \times 1 \times 0! = 3 \times 2 \times 1 \times 1 = 6$

Função Fatorial Recursiva

```
1  # Função recursiva para cálculo do fatorial
2  def fat(n):
3      if n == 0: # condição de parada
4          return 1
5      else:
6          return n * fat(n - 1) # chamada recursiva
7
8  # Programa principal
9  x = fat(3)
10 print(x)
```

Saída:

Ativação de Funções

- Durante a execução de um **programa recursivo**:
 - ▶ Uma área especial de memória, organizada em forma de **pilha (stack)**, é utilizada para armazenar:
 - ▶ Valores das variáveis locais;
 - ▶ Parâmetros das funções.
 - ▶ Outra área é organizada para manter as variáveis globais.
- **Pilha**: É uma estrutura de dados do tipo **LIFO** (Last In, First Out), em que o acesso é sempre feito **somente ao topo**.
 - ▶ **Empilhar** adiciona um elemento no topo da pilha;
 - ▶ **Desempilhar** remove o elemento do topo.
- Quando o programa principal é iniciado, cria-se espaço para variáveis globais.
- Sempre que uma função é ativada:
 - ▶ Um espaço contendo variáveis locais e parâmetros é reservado no **topo da pilha**.
- Ao fim da função:
 - ▶ Seu espaço é removido do topo da pilha e pode ser reutilizado.

Execução de $\text{fat}(3)$ na Pilha

Chamadas recursivas e estado da pilha:

Topo da Pilha	Conteúdo
$\text{fat}(0)$	retorna 1
$\text{fat}(1)$	1×1
$\text{fat}(2)$	$2 \times \text{fat}(1)$
$\text{fat}(3)$	$3 \times \text{fat}(2)$

Ordem de execução:

1. $\text{fat}(3)$ chama $\text{fat}(2)$
2. $\text{fat}(2)$ chama $\text{fat}(1)$
3. $\text{fat}(1)$ chama $\text{fat}(0)$
4. $\text{fat}(0)$ retorna 1 (caso base)
5. Desempilha e calcula:

$$1 \rightarrow (1 \times 1) \rightarrow (2 \times 1) \rightarrow (3 \times 2)$$

Resultado final: 6

Execução Visual de $\text{fat}(3)$ na Pilha

Crescimento da Pilha (chamadas recursivas):

$\text{fat}(3): 3 \times \text{fat}(2)$
$\text{fat}(2): 2 \times \text{fat}(1)$
$\text{fat}(1): 1 \times \text{fat}(0)$
$\text{fat}(0): \text{retorna } 1$

Desempilhando (retornos):

$$\text{fat}(0) = 1 \quad \Rightarrow \quad \text{fat}(1) = 1 \times 1 = 1$$

$$\text{fat}(2) = 2 \times 1 = 2 \quad \Rightarrow \quad \text{fat}(3) = 3 \times 2 = 6$$

Resultado final: 6

Recursividade Mútua

- Em alguns casos, dois subprogramas podem se chamar reciprocamente.
- Esse tipo de técnica é chamado de **recursividade mútua**.

```
1 # Subprogramas
2 def flip(n):
3     print("Flip", end = " ")
4     if n > 0:
5         flop(n - 1)
6
7 def flop(n):
8     print("Flop", end = " ")
9     if n > 0:
10        flip(n - 1)
11
12 # Programa Principal
13 flip(5)
```

Saída:

Flip Flop Flip Flop Flip Flop

Execução Visual — Recursividade Mútua

Chamadas recursivas de flip(5):

flip(5): imprime "Flip" → chama flop(4)

flop(4): imprime "Flop" → chama flip(3)

flip(3): imprime "Flip" → chama flop(2)

flop(2): imprime "Flop" → chama flip(1)

flip(1): imprime "Flip" → chama flop(0)

flop(0): imprime "Flop" e encerra

Flip → Flop → Flip → Flop → Flip → Flop

Obs.: A pilha alterna chamadas entre as duas funções até atingir a condição de parada ($n = 0$).

Algoritmo Iterativo vs Recursivo

Exemplo: Algoritmo Iterativo — Cálculo do Fatorial

```
1 def fatorial_iterativo(n):  
2     fat = 1  
3     for i in range(1, n+1):  
4         fat *= i  
5     return fat  
6  
7 # Programa Principal  
8 print(fatorial_iterativo(5)) # Saída: 120
```

Vantagens:

- Não utiliza pilha de chamadas adicional.
- Simples de implementar.

Complexidade: $O(n)$

Algoritmo Iterativo vs Recursivo

Exemplo: Algoritmo Recursivo — Cálculo do Fatorial

```
1 def fatorial_recursivo(n):  
2     if n == 0: # caso base  
3         return 1  
4     else:  
5         return n * fatorial_recursivo(n - 1)  
6  
7 # Programa Principal  
8 print(fatorial_recursivo(5)) # Saída: 120
```

Observação:

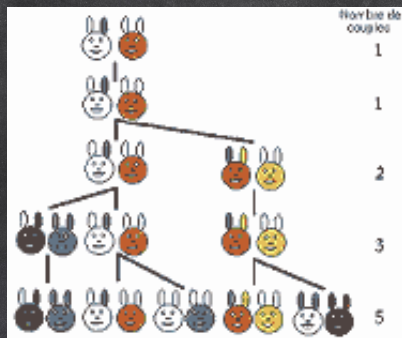
- Usa chamadas aninhadas na pilha.
- Melhor para expressar soluções que dependem de subdivisão natural.

Complexidade: $O(n)$, mas com sobrecarga de chamadas recursivas.

Sequência de Fibonacci

- Famoso problema proposto por Leonardo de Pisa em 1202.
- Objetiva-se determinar o número de pares de coelhos ao final de 12 meses sob as seguintes condições:
 - ▶ Inicialmente tem-se um único par de coelhos (um macho e uma fêmea) recém nascidos.
 - ▶ Todo mês, cada par de coelhos com mais de 1 mês produz um novo par de coelhos de sexos opostos.
 - ▶ Nenhum coelho morre durante esse processo.

Exemplo



Um pouco mais de detalhes

Mês	Qtd	Obs
1	1	início do mês
2	1	casal do mês 1 não procriou
3	$1 + 1 = 2$	1 do mês 2 + 1 novo pra cada do mês 1
4	$2 + 1 = 3$	2 do mês 3 + 1 novo pra cada do mês 2
5	$3 + 2 = 5$	3 do mês 4 + 1 novo pra cada do mês 3
6	$5 + 3 = 8$	5 do mês 5 + 1 novo pra cada do mês 4

Definição recursiva

Seja $\{F_n\}$ definida **recursivamente** da seguinte forma:

$$\begin{cases} F_0 = 1 \\ F_1 = 1 \\ F_n = F_{n-1} + F_{n-2}, n \geq 2 \end{cases}$$

```
1 # Função recursiva para Fibonacci
2 def fibonacci(n):
3     if n == 0:                # caso base
4         return 1
5     elif n == 1:             # caso base
6         return 1
7     else:                    # caso recursivo
8         return fibonacci(n-1) + fibonacci(n-2)
9
10 # Programa Principal
11 for i in range(10):
12     print(fibonacci(i), end=" ")
13 # Saída: 1 1 2 3 5 8 13 21 34 55
```

Complexidade: $O(2^n)$

Comparação iterativo vs recursivo da sequência de Fibonacci

Duas versões para a obtenção do n -ésimo termo:

```
1  n = int(input())
2
3  def fibonacci_iterativo(n):
4      lista = []
5
6      lista.append(1)
7      lista.append(1)
8
9      for i in range(2, n+1):
10         lista.append(lista[i-1] + lista[i-2])
11
12     return lista[n]
13
14 def fibonacci_recursivo(n):
15     if n == 0:                # caso base
16         return 1
17     elif n == 1:              # caso base
18         return 1
19     else:                      # caso recursivo
20         return fibonacci_recursivo(n-1) + fibonacci_recursivo(n-2)
21
22 print(fibonacci_iterativo(n))
23 print(fibonacci_recursivo(n))
```

Complexidade da versão iterativa $O(n)$, pois para obter cada elemento fazemos soma dos elementos nas duas posições anteriores.

Seja $n = 100$, teste a versão iterativa, depois teste com a versão recursiva. Note a diferença de tempo de processamento.

Torres de Hanói

Configuração inicial: n discos de diâmetros diferentes estão empilhados no pino A, do maior na base ao menor no topo.

Objetivo: Transferir todos os discos do pino A para o pino C, utilizando o pino B como auxiliar.

Regras:

1. Apenas o disco do topo de um pino pode ser movido.
2. Nunca um disco de diâmetro maior pode ser colocado sobre um disco de diâmetro menor.

Solução $T(n)$ para n discos: Recursivamente: transfira os $n - 1$ discos superiores de A para B, utilizando C como auxiliar (isso é feito com $T(n - 1)$ movimentos); transfira o disco que permaneceu em A para B (isso é feito com 1 movimento); transfira os $n - 1$ discos de B para C, utilizando A como auxiliar (isso é feito com $T(n - 1)$ movimentos).

Número mínimo de movimentos para n discos:

$$T(n) = 2T(n - 1) + 1, \quad T(1) = 1.$$

Algoritmo Recursivo: Torre de Hanói

Problema: Obter sequência de movimentos dos n discos do pino A para o pino C usando o pino B como auxiliar.

```
1 def hanoi(n, origem, destino, auxiliar):
2     if n > 0:
3         # mover n-1 discos para torre auxiliar
4         hanoi(n-1, origem, auxiliar, destino)
5         # mover disco restante para destino
6         print(f"Mover disco de {origem} para {destino}")
7         # mover n-1 discos da torre auxiliar para destino
8         hanoi(n-1, auxiliar, destino, origem)
9
10 # Programa Principal
11 hanoi(3, "A", "C", "B")
```

Complexidade (quantidade de movimentos): $O(2^n - 1)$.

Soma de Elementos de uma Lista

Dada uma lista v de inteiros, calcular a soma dos elementos das posições 0 até $tam - 1$, onde tam é o tamanho da lista.

Definição Recursiva:

$$S(n) = \begin{cases} v[0], & \text{se } n = 0 \quad (\text{caso base}) \\ v[n] + S(n-1), & \text{se } n > 0 \quad (\text{passo recursivo}) \end{cases}$$

```
1 # Função recursiva para soma
2 def soma(v, n):
3     if n == 0: # caso base
4         return v[0]
5     else:
6         return v[n] + soma(v, n-1)
7
8 # Programa Principal
9 vet = [1, 2, 3, 4, 5]
10 print("A soma dos elementos de vet é: %d" % soma(vet, len(vet)-1))
11 # Saída: A soma dos elementos de vet é: 15
```

Obs.: Algoritmos recursivos tendem a ser mais claros e concisos, mas podem ter sobrecarga de execução em relação aos iterativos.